

IoT Assignments - Week 1

18/06/26

- simulate an array of leds in TinkerCAD which fade in - fade out one by one.
https://www.tinkercad.com/things/bykP8v0A084-led-fade-array?sharecode=PjNu2ATi2N720VUWKqGk_50Avil8xY_MZN_FXvNw2LU
- simulate a 4x4 led matrix in TinkerCAD, every number should have an led corresponding to it. play around with it (eg. display your initials)
array:
https://www.tinkercad.com/things/5st4dv4BNnc-4x4-using-array?sharecode=a3iKWJEVH7OaRgw92IsMvMeYP2-Y_jEjAHhLKItXGoE
matrix:
<https://www.tinkercad.com/things/2A2ARfAASSv-4x4-using-matrix?sharecode=dXuMuSWGff-Zx0TzCKy5rMadOiUBIjEnZGCHO853vW8>
- research about the creation, interface and variations of the seven segment display and tinker around with it.
Here, i created dice using 7 segment display
<https://www.tinkercad.com/things/eucAGBSPh9e-dice?sharecode=ILMGCMJuoN SPPP9CUc3OQnMpfqskWZUXv2CM0oS7WzI>

21/06/26

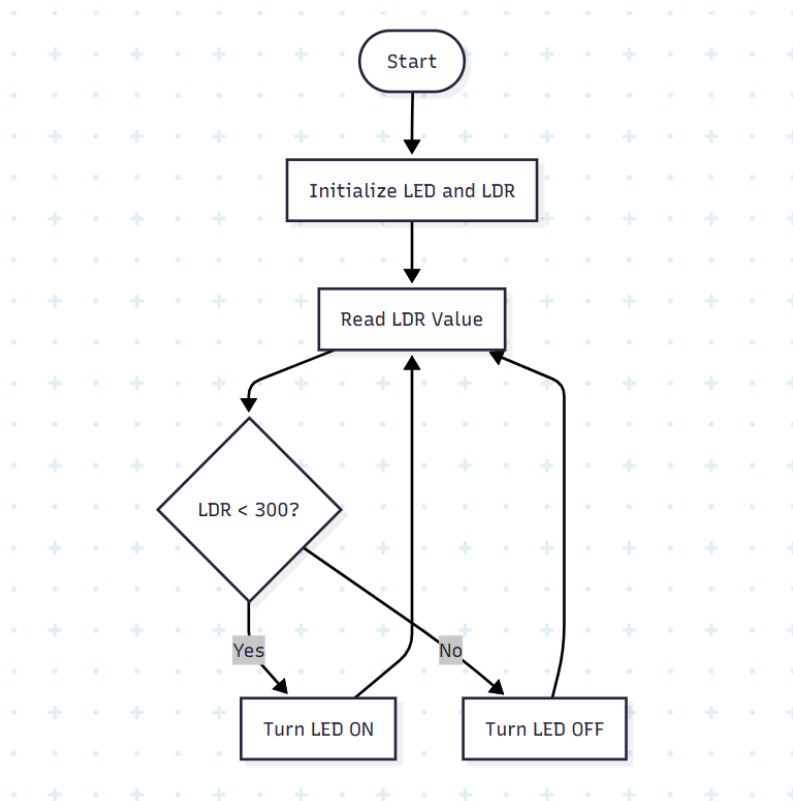
- Design and build a Reaction Time Game using an Arduino, 3 LEDs, and 1 push button. When the game starts, the three LEDs should blink in a random order and stay ON for random durations to create an unpredictable waiting period. After the random LED sequence is completed, the green LED should turn ON as the signal for the player to react. The player must press the push button as quickly as possible after the green LED lights up. The Arduino should measure the player's reaction time and display the result in milliseconds (ms) on the Serial Monitor.
https://www.tinkercad.com/things/8gW5UgFPHc4-reaction-time-game?sharecode=7Jq4pOxRMI3OKRdamhR0lu2RkNtMp0n6T_zQDVU-DAC

- Write an algorithm/flowchart and C++ program for a smart street light system with the following requirements:
 - * Read an LDR sensor value.
 - * If the value is less than 300, turn the LED ON.
 - * Otherwise, turn the LED OFF.
 - * Repeat indefinitely.

Algorithm

1. Start
2. Initialize LED pin as OUTPUT.
3. Read value from LDR sensor.
4. If LDR value < 300:
 - Turn LED ON
5. Else:
 - Turn LED OFF.
6. Go to step 3.

Flowchart



Program

```
int ldr = 7;

int led = 5;

void setup(){
  pinMode(led, OUTPUT);}

void loop(){
  int ldrval = analogRead(ldr);
  if(ldrValue < 300){
    digitalWrite(ledPin, HIGH);}
  else{
    digitalWrite(ledPin, LOW);}
  delay(100);}
```

- An IoT system contains 4 sensor nodes arranged in rows and columns. Using nested loops, print:
- Sensor(1,1) Sensor(1,2) Sensor(1,3)
- Sensor(2,1) Sensor(2,2) Sensor(2,3)
- Sensor(3,1) Sensor(3,2) Sensor(3,3)
- Sensor(4,1) Sensor(4,2) Sensor(4,3)

```
void setup(){
  Serial.begin(9600);
  for(int row = 1; row <= 4; row++){
    for(int col = 1; col <= 3; col++){
      Serial.print("Sensor(");
      Serial.print(row);
      Serial.print(",");
      Serial.print(col);
      Serial.print(") ");
    }
    Serial.println();
  }
  void loop(){}
```

- note down differences between if-else and switch-case. Where can they be used in IoT?

The **if-else statement** is used when conditions involve comparisons such as greater than, less than, equal to, or multiple logical conditions. It is more flexible and can handle complex decision-making. For example, in an IoT-based smart street light system, an if-else statement can be used to check whether the LDR sensor value is below a certain threshold and turn the street light ON or OFF accordingly.

On the other hand, **switch-case** is used when a variable can have a fixed set of values. It makes the code cleaner and easier to read when there are many options to choose from. For example, in an IoT home automation system, switch-case can be used to select different operating modes such as Home Mode, Away Mode, or Sleep Mode.

In general, if-else is preferred for sensor-based decisions and complex conditions, while switch-case is preferred for selecting among multiple predefined options or system states.

if-else

- Smart street lighting systems
- Temperature monitoring systems
- Smart irrigation systems
- Gas leakage detection systems

switch-case

- Home automation modes
 - Robot movement control
 - Device operating modes
 - Smart appliance control systems
-
- what are system states and how to manage them?

A system state represents the current operating condition of a system. State management involves controlling transitions between different states based on inputs, events, or sensor readings. In IoT systems, states are commonly managed using if-else statements, switch-case statements, or finite state machines to ensure organized and efficient system behavior.